

```

In [4]: # =====
# FULL ONE-CELL AUDIO EDA (BASIC + ADVANCED) – Jupyter Local
# =====

# --- Installing dependencies ---
import sys, subprocess
def _pip_install(pkgs):
    subprocess.run([sys.executable, "-m", "pip", "install", "-q"] + pkgs, check=
_pip_install([
    "librosa", "soundfile", "pandas", "numpy", "matplotlib", "seaborn",
    "tqdm", "scikit-learn", "scipy"
]))

# --- Importing Libraries ---
import os, glob, warnings
from pathlib import Path
import numpy as np
import pandas as pd
from tqdm.notebook import tqdm

import librosa
import librosa.display
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

import scipy.signal as signal
from scipy.stats import ttest_ind
from sklearn.ensemble import RandomForestClassifier
from IPython.display import display

warnings.filterwarnings("ignore")
sns.set_style("whitegrid")
plt.rcParams["figure.figsize"] = (10, 4)

# =====
# PATHS + CONFIG
# =====

# --- Resolving base directory relative to notebook location ---
BASE_DIR = Path(os.getcwd())

METADATA_PATH      = BASE_DIR / "metadata_handover.csv"
AUDIO_DIR           = BASE_DIR / "audio_files"
EDA_OUTPUT_DIR      = BASE_DIR / "eda_outputs"
EDA_OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

SR                  = 16000
TOP_DB_TRIM         = 25
N_MFCC              = 20

STATS_SAMPLE_N      = 2500
EMBED_SAMPLE_N      = 1200
AVG_MFCC_PER_CLASS_N = 200

SAVE_PLOTS          = True

```

```

PAIRWISE_TEST_FEATURES = ["rms", "zcr", "centroid", "rolloff"]
PAIRWISE_LABELS        = None

print("Base directory :", BASE_DIR)
print("Metadata       :", METADATA_PATH)
print("Audio directory:", AUDIO_DIR)
print("Outputs        :", EDA_OUTPUT_DIR)

# =====
# UTILITIES
# =====

def savefig(name):
    if SAVE_PLOTS:
        out = EDA_OUTPUT_DIR / name
        plt.savefig(out, dpi=200, bbox_inches="tight")
        print("Saved:", out)

# =====
# DETECTING AUDIO FILES
# =====

if not AUDIO_DIR.exists():
    raise FileNotFoundError(
        f"Audio folder not found at: {AUDIO_DIR}\n"
        f"Make sure your audio folder is named 'audio_files' and placed in: {BAS
    )

# --- Detecting audio files recursively ---
audio_files = []
for ext in [".wav", ".mp3", ".flac", ".ogg", ".m4a"]:
    audio_files += glob.glob(str(AUDIO_DIR / "*" / ext), recursive=True)

print(f"Audio files found: {len(audio_files)}")
if len(audio_files) == 0:
    raise FileNotFoundError(
        f"No audio files found in: {AUDIO_DIR}\n"
        "Supported formats: .wav, .mp3, .flac, .ogg, .m4a"
    )
print("Example files:", audio_files[:3])

# =====
# LOADING METADATA
# =====

if not METADATA_PATH.exists():
    # --- Searching for any CSV in base directory as fallback ---
    csv_candidates = list(BASE_DIR.glob("*.csv"))
    if csv_candidates:
        METADATA_PATH = csv_candidates[0]
        print(f"metadata_handover.csv not found. Using fallback: {METADATA_PATH}")
    else:
        raise FileNotFoundError(
            f"No metadata CSV found in: {BASE_DIR}\n"
            "Place your metadata CSV in the same folder as the notebook."
        )

df = pd.read_csv(METADATA_PATH)
print("\nLoaded metadata:", df.shape)

```

```

print("Columns:", df.columns.tolist())
print(df.head(3))

# =====
# BUILDING / REPAIRING file_path
# =====

def resolve_audio_path(fid):
    """Resolving audio file path by searching for matching filename."""
    fid = str(fid)
    # Direct match
    p = AUDIO_DIR / fid
    if p.exists():
        return str(p)
    # Try appending extensions
    for ext in [".wav", ".mp3", ".flac", ".ogg", ".m4a"]:
        p2 = AUDIO_DIR / (fid + ext)
        if p2.exists():
            return str(p2)
    # Recursive search by filename stem
    for ext in ["wav", "mp3", "flac", "ogg", "m4a"]:
        matches = list(AUDIO_DIR.rglob(f"{Path(fid).stem}.{ext}"))
        if matches:
            return str(matches[0])
    return str(p)

if "file_path" not in df.columns:
    if "file_id" in df.columns:
        df["file_path"] = df["file_id"].apply(resolve_audio_path)
        print("Created 'file_path' from 'file_id'.")
    else:
        raise ValueError("Metadata must contain either 'file_path' or 'file_id'.")
else:
    # --- Repairing absolute Colab paths to local paths ---
    def repair_path(p):
        p = str(p)
        filename = Path(p).name
        # Try direct match in audio dir
        local = AUDIO_DIR / filename
        if local.exists():
            return str(local)
        # Try recursive search
        for ext in ["wav", "mp3", "flac", "ogg", "m4a"]:
            matches = list(AUDIO_DIR.rglob(f"{Path(filename).stem}.{ext}"))
            if matches:
                return str(matches[0])
        return str(local)

    # Detect if paths look like Colab/absolute paths
    sample_path = str(df["file_path"].iloc[0])
    if not Path(sample_path).exists():
        print("Detected broken/absolute paths – repairing to local paths...")
        df["file_path"] = df["file_path"].apply(repair_path)
        print("Paths repaired.")

df["file_path"] = df["file_path"].astype(str)
df["label"] = df["label"].astype(str)

# =====
# VALIDATING FILE PATHS

```

```

# =====

df["file_exists"] = df["file_path"].apply(lambda p: Path(p).exists())
missing_files = int((~df["file_exists"]).sum())
print(f"\nMissing audio files: {missing_files} / {len(df)}")

if missing_files > 0:
    missing_df = df[~df["file_exists"]][["file_path", "label"]].copy()
    missing_df.to_csv(EDA_OUTPUT_DIR / "missing_files.csv", index=False)
    print("Saved missing file list to:", EDA_OUTPUT_DIR / "missing_files.csv")
    print("Sample of missing paths:")
    print(missing_df["file_path"].head(5).tolist())

df = df[df["file_exists"]].reset_index(drop=True)
print("After removing missing files:", df.shape)

if len(df) == 0:
    raise ValueError(
        "No valid audio files matched. Check that:\n"
        f" 1. Audio files are inside: {AUDIO_DIR}\n"
        " 2. Filenames in metadata match actual files on disk."
    )

# =====
# BASIC EDA - DATASET STATISTICS
# =====

print("\n=====")
print("BASIC EDA (1) DATASET STATISTICS")
print("=====")
print("Total samples:", len(df))
print("Classes:", sorted(df["label"].unique()))

class_counts = df["label"].value_counts().reset_index()
class_counts.columns = ["label", "count"]
display(class_counts)

imb_ratio = class_counts["count"].max() / class_counts["count"].min()
print("Imbalance ratio (max/min):", round(float(imb_ratio), 2))

plt.figure(figsize=(10, 4))
sns.barplot(data=class_counts, x="label", y="count")
plt.title("Class Distribution")
plt.xticks(rotation=20)
plt.tight_layout()
savefig("class_distribution.png")
plt.show()

# --- Plotting categorical distributions ---
for col in ["dataset_source", "device_type"]:
    if col in df.columns:
        tmp = df[col].value_counts(dropna=False).reset_index()
        tmp.columns = [col, "count"]
        print(f"\nDistribution of {col}:")
        display(tmp.head(20))
        plt.figure(figsize=(10, 4))
        sns.barplot(data=tmp.head(20), x=col, y="count")
        plt.title(f"{col} distribution (top 20)")
        plt.xticks(rotation=30)
        plt.tight_layout()
        savefig(f"{col}_distribution_top20.png")

```

```

plt.show()

# --- Saving device type percentages ---
if "device_type" in df.columns:
    device_pct = (df["device_type"].value_counts(normalize=True) * 100).round(2)
    print("\nDevice type %:")
    display(device_pct)
    device_pct.to_csv(EDA_OUTPUT_DIR / "device_type_percent.csv")
    print("Saved:", EDA_OUTPUT_DIR / "device_type_percent.csv")

# =====
# AUDIO HELPERS
# =====

def safe_load(path, sr=SR):
    try:
        y, _sr = librosa.load(path, sr=sr, mono=True)
        if y is None or len(y) == 0:
            return None, None
        return y, _sr
    except Exception:
        return None, None

def compute_basic_audio_stats(path):
    y, sr = safe_load(path)
    if y is None:
        return {"duration_sec": np.nan, "rms": np.nan, "zcr": np.nan, "silence_r
    y, _ = librosa.effects.trim(y, top_db=TOP_DB_TRIM)
    duration = len(y) / sr
    rms = float(np.sqrt(np.mean(y**2))) if len(y) else np.nan
    zcr = float(np.mean(librosa.feature.zero_crossing_rate(y))) if len
    thr = 0.01
    silence_ratio = float(np.mean(np.abs(y) < thr)) if len(y) else np.nan
    return {"duration_sec": duration, "rms": rms, "zcr": zcr, "silence_ratio": s

def plot_waveform(y, sr, title):
    plt.figure(figsize=(10, 3))
    librosa.display.waveshow(y, sr=sr)
    plt.title(title)
    plt.tight_layout()

def plot_spectrogram(y, sr, title):
    S = librosa.stft(y, n_fft=1024, hop_length=256)
    S_db = librosa.amplitude_to_db(np.abs(S), ref=np.max)
    plt.figure(figsize=(10, 4))
    librosa.display.specshow(S_db, sr=sr, hop_length=256, x_axis="time", y_axis=
    plt.colorbar(format="+2.0f dB")
    plt.title(title)
    plt.tight_layout()

def plot_mfcc(y, sr, title, n_mfcc=N_MFCC):
    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=n_mfcc)
    plt.figure(figsize=(10, 4))
    librosa.display.specshow(mfcc, x_axis="time", sr=sr)
    plt.colorbar()
    plt.title(title)
    plt.tight_layout()

# =====
# BASIC EDA – AUDIO-LEVEL STATISTICS

```

```

# =====
print("\n=====")
print("BASIC EDA (2) AUDIO-LEVEL STATS")
print("=====")

stats_n = min(len(df), STATS_SAMPLE_N)
df_stats = df.sample(stats_n, random_state=7).copy()

rows, bad_loads = [], 0
for p in tqdm(df_stats["file_path"], total=len(df_stats), desc="Computing basic
    st = compute_basic_audio_stats(p)
    if np.isnan(st["duration_sec"]):
        bad_loads += 1
    rows.append(st)

stats_df = pd.DataFrame(rows)
df_stats = pd.concat([df_stats.reset_index(drop=True), stats_df], axis=1)

print(f"Bad/failed loads: {bad_loads}/{len(df_stats)}")
display(df_stats[["duration_sec", "rms", "zcr", "silence_ratio"]].describe())

# --- Plotting basic feature correlation ---
plt.figure(figsize=(6, 5))
sns.heatmap(df_stats[["duration_sec", "rms", "zcr", "silence_ratio"]].corr(), an
plt.title("Basic Feature Correlation")
plt.tight_layout()
savefig("basic_feature_correlation.png")
plt.show()

# --- Plotting per-class boxplots ---
for feat, title, fname in [
    ("duration_sec", "Duration (sec) by class (sampled)", "duration_by_class_b
    ("rms", "RMS Energy by class (sampled)", "rms_by_class_boxpl
    ("silence_ratio", "Silence ratio by class (sampled)", "silence_ratio_by_c
]:
    plt.figure(figsize=(10, 4))
    sns.boxplot(data=df_stats, x="label", y=feat)
    plt.title(title)
    plt.xticks(rotation=20)
    plt.tight_layout()
    savefig(fname)
    plt.show()

# --- Plotting duration vs RMS scatter ---
plt.figure(figsize=(6, 5))
plt.scatter(df_stats["duration_sec"], df_stats["rms"], alpha=0.3)
plt.xlabel("Duration (sec)")
plt.ylabel("RMS Energy")
plt.title("Duration vs RMS (Outlier Detection)")
plt.tight_layout()
savefig("duration_vs_rms_scatter.png")
plt.show()

# --- Identifying outlier samples ---
quiet = df_stats.nsmallest(5, "rms")[["file_path", "label", "rms", "duration_sec",
loud = df_stats.nlargest(5, "rms")[["file_path", "label", "rms", "duration_sec",
short = df_stats.nsmallest(5, "duration_sec")[["file_path", "label", "rms", "durat
long_ = df_stats.nlargest(5, "duration_sec")[["file_path", "label", "rms", "durati
silent = df_stats.nlargest(5, "silence_ratio")[["file_path", "label", "rms", "durat

```

```

weird = pd.concat([short, long_, quiet, loud, silent], axis=0).reset_index(drop=
print("\nInteresting / outlier samples:")
display(weird)
weird.to_csv(EDA_OUTPUT_DIR / "weird_samples.csv", index=False)
print("Saved:", EDA_OUTPUT_DIR / "weird_samples.csv")

# =====
# BASIC EDA – EXAMPLE VISUALS PER CLASS
# =====
print("\n=====")
print("BASIC EDA (3) EXAMPLE VISUALS PER CLASS")
print("=====")

for lab in sorted(df["label"].unique()):
    sample = df[df["label"] == lab].sample(1, random_state=42).iloc[0]
    path = sample["file_path"]
    y, sr = safe_load(path)
    if y is None:
        print(f"Could not load example for {lab}: {path}")
        continue

    y, _ = librosa.effects.trim(y, top_db=TOP_DB_TRIM)
    base_name = lab.replace(" ", "_").lower()
    print(f"\n=== {lab} === File: {Path(path).name}")

    plot_waveform(y, sr, f"Waveform | {lab}")
    savefig(f"example_{base_name}_waveform.png")
    plt.show()

    plot_spectrogram(y, sr, f"Spectrogram | {lab}")
    savefig(f"example_{base_name}_spectrogram.png")
    plt.show()

    plot_mfcc(y, sr, f"MFCC (n={N_MFCC}) | {lab}")
    savefig(f"example_{base_name}_mfcc.png")
    plt.show()

# =====
# BASIC EDA – DOMAIN SHIFT CHECKS
# =====
print("\n=====")
print("BASIC EDA (4) DOMAIN SHIFT CHECKS")
print("=====")

# --- Plotting dataset_source vs Label heatmap ---
if "dataset_source" in df.columns:
    pivot = pd.crosstab(df["dataset_source"], df["label"])
    print("\nCounts: dataset_source vs label")
    display(pivot)
    display(pivot.div(pivot.sum(axis=1), axis=0))

    plt.figure(figsize=(10, 5))
    sns.heatmap(pivot, annot=True, fmt="d", cmap="Blues")
    plt.title("Heatmap: dataset_source vs label")
    plt.tight_layout()
    savefig("heatmap_dataset_source_vs_label.png")
    plt.show()

# --- Plotting device_type vs Label heatmap ---
if "device_type" in df.columns:

```

```

pivot2 = pd.crosstab(df["device_type"], df["label"])
print("\nCounts: device_type vs label")
display(pivot2)
display(pivot2.div(pivot2.sum(axis=1), axis=0))

plt.figure(figsize=(10, 5))
sns.heatmap(pivot2, annot=True, fmt="d", cmap="Greens")
plt.title("Heatmap: device_type vs label")
plt.tight_layout()
savefig("heatmap_device_type_vs_label.png")
plt.show()

# =====
# BASIC EDA - MFCC-MEAN EMBEDDINGS (PCA + t-SNE)
# =====
print("\n=====")
print("BASIC EDA (5) FEATURE EMBEDDINGS (MFCC-mean)")
print("=====")

def mfcc_mean_feature(path, n_mfcc=N_MFCC):
    y, sr = safe_load(path)
    if y is None:
        return None
    y, _ = librosa.effects.trim(y, top_db=TOP_DB_TRIM)
    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=n_mfcc)
    return mfcc.mean(axis=1)

embed_n = min(len(df), EMBED_SAMPLE_N)
sub = df.sample(embed_n, random_state=3).reset_index(drop=True)

X, keep = [], []
for i, p in enumerate(tqdm(sub["file_path"], desc="Extracting MFCC-mean features")):
    feat = mfcc_mean_feature(p)
    if feat is not None and np.all(np.isfinite(feat)):
        X.append(feat)
        keep.append(i)

if len(X) < 10:
    print("Not enough valid features extracted for PCA/t-SNE.")
else:
    X = np.vstack(X)
    sub = sub.iloc[keep].reset_index(drop=True)
    print("Feature matrix:", X.shape)

    # --- Running PCA on MFCC-mean features ---
    pca = PCA(n_components=2, random_state=0)
    X2 = pca.fit_transform(X)

    plt.figure(figsize=(8, 6))
    for lab in sorted(sub["label"].unique()):
        idx = (sub["label"].values == lab)
        plt.scatter(X2[idx, 0], X2[idx, 1], s=12, label=lab)
    plt.title("PCA on MFCC-mean features (by label)")
    plt.legend()
    plt.tight_layout()
    savefig("pca_mfcc_mean_by_label.png")
    plt.show()

    # --- Running t-SNE on MFCC-mean features ---
    tsne = TSNE(n_components=2, perplexity=30, init="pca", learning_rate="auto",

```

```

Xts = tsne.fit_transform(X)

plt.figure(figsize=(8, 6))
for lab in sorted(sub["label"].unique()):
    idx = (sub["label"].values == lab)
    plt.scatter(Xts[idx, 0], Xts[idx, 1], s=12, label=lab)
plt.title("t-SNE on MFCC-mean features (by label)")
plt.legend()
plt.tight_layout()
savefig("tsne_mfcc_mean_by_label.png")
plt.show()

# --- Plotting PCA colored by dataset source ---
if "dataset_source" in sub.columns:
    plt.figure(figsize=(8, 6))
    for src in sorted(sub["dataset_source"].astype(str).unique())[:12]:
        idx = (sub["dataset_source"].astype(str).values == src)
        plt.scatter(X2[idx, 0], X2[idx, 1], s=12, label=src)
    plt.title("PCA on MFCC-mean features (by dataset_source)")
    plt.legend(bbox_to_anchor=(1.05, 1), loc="upper left")
    plt.tight_layout()
    savefig("pca_mfcc_mean_by_dataset_source.png")
    plt.show()

# =====
# ADVANCED EDA – SPECTRAL FEATURES
# =====
print("\n=====")
print("ADVANCED EDA (1) SPECTRAL FEATURES")
print("=====")

def spectral_features(path):
    y, sr = safe_load(path)
    if y is None:
        return {"centroid": np.nan, "bandwidth": np.nan, "rolloff": np.nan, "flatness": np.nan}
    y, _ = librosa.effects.trim(y, top_db=TOP_DB_TRIM)
    return {
        "centroid": float(np.mean(librosa.feature.spectral_centroid(y=y, sr=sr))),
        "bandwidth": float(np.mean(librosa.feature.spectral_bandwidth(y=y, sr=sr))),
        "rolloff": float(np.mean(librosa.feature.spectral_rolloff(y=y, sr=sr))),
        "flatness": float(np.mean(librosa.feature.spectral_flatness(y=y))),
    }

# --- Computing spectral features ---
spectral_rows = []
for p in tqdm(df_stats["file_path"], desc="Computing spectral features"):
    spectral_rows.append(spectral_features(p))

spectral_df = pd.DataFrame(spectral_rows)
df_stats = pd.concat([df_stats.reset_index(drop=True), spectral_df], axis=1)
print("Added spectral features to df_stats.")

# --- Plotting spectral feature boxplots per class ---
for feat in ["centroid", "bandwidth", "rolloff", "flatness"]:
    plt.figure(figsize=(10, 4))
    sns.boxplot(data=df_stats, x="label", y=feat)
    plt.title(f"{feat} by class (sampled)")
    plt.xticks(rotation=20)
    plt.tight_layout()
    savefig(f"{feat}_by_class_boxplot.png")

```

```

plt.show()

# =====
# ADVANCED EDA – FULL FEATURE CORRELATION
# =====
print("\n=====")
print("ADVANCED EDA (2) FULL FEATURE CORRELATION")
print("=====")

feature_cols = ["duration_sec", "rms", "zcr", "silence_ratio",
                "centroid", "bandwidth", "rolloff", "flatness"]
corr = df_stats[feature_cols].corr()

plt.figure(figsize=(9, 7))
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Full Feature Correlation Matrix")
plt.tight_layout()
savefig("full_feature_correlation_matrix.png")
plt.show()

# =====
# ADVANCED EDA – PAIRWISE STATISTICAL TESTS
# =====
print("\n=====")
print("ADVANCED EDA (3) STATISTICAL TESTS (PAIRWISE)")
print("=====")

labels_for_tests = PAIRWISE_LABELS
if labels_for_tests is None:
    labels_for_tests = class_counts["label"].head(4).tolist()
print("Testing labels :", labels_for_tests)
print("Testing features:", PAIRWISE_TEST_FEATURES)

# --- Running pairwise t-tests ---
test_rows = []
for i in range(len(labels_for_tests)):
    for j in range(i + 1, len(labels_for_tests)):
        a, b = labels_for_tests[i], labels_for_tests[j]
        sub_a = df_stats[df_stats["label"] == a]
        sub_b = df_stats[df_stats["label"] == b]
        for feat in PAIRWISE_TEST_FEATURES:
            if feat not in df_stats.columns:
                continue
            x = sub_a[feat].dropna()
            y = sub_b[feat].dropna()
            if len(x) < 5 or len(y) < 5:
                continue
            stat, p = ttest_ind(x, y, equal_var=False)
            test_rows.append({"class_A": a, "class_B": b, "feature": feat, "p_value": p})

tests_df = pd.DataFrame(test_rows).sort_values("p_value")
display(tests_df.head(30))
tests_df.to_csv(EDA_OUTPUT_DIR / "pairwise_ttests.csv", index=False)
print("Saved:", EDA_OUTPUT_DIR / "pairwise_ttests.csv")

# =====
# ADVANCED EDA – DOMAIN SHIFT FEATURE COMPARISON
# =====
print("\n=====")
print("ADVANCED EDA (4) DOMAIN SHIFT FEATURE COMPARISON")

```

```

print("=====")

# --- Comparing features across device types ---
if "device_type" in df_stats.columns:
    for feat in ["rms", "centroid", "rolloff"]:
        plt.figure(figsize=(8, 4))
        sns.boxplot(data=df_stats, x="device_type", y=feat)
        plt.title(f"{feat} by device_type (sampled)")
        plt.xticks(rotation=30)
        plt.tight_layout()
        savefig(f"{feat}_by_device_type_boxplot.png")
        plt.show()

# =====
# ADVANCED EDA – RANDOM FOREST FEATURE IMPORTANCE
# =====
print("\n=====")
print("ADVANCED EDA (5) QUICK FEATURE IMPORTANCE (RandomForest)")
print("=====")

# --- Training RandomForest for feature importance ---
rf_X = df_stats[feature_cols].fillna(0)
rf_y = df_stats["label"]

rf = RandomForestClassifier(n_estimators=200, random_state=42)
rf.fit(rf_X, rf_y)

importance = pd.Series(rf.feature_importances_, index=feature_cols).sort_values(
    display(importance.to_frame("importance")))

plt.figure(figsize=(9, 4))
importance.plot(kind="bar")
plt.title("Random Forest Feature Importance (EDA only)")
plt.tight_layout()
savefig("rf_feature_importance.png")
plt.show()

importance.to_csv(EDA_OUTPUT_DIR / "rf_feature_importance.csv")
print("Saved:", EDA_OUTPUT_DIR / "rf_feature_importance.csv")

# =====
# ADVANCED EDA – AVERAGE MFCC PER CLASS
# =====
print("\n=====")
print("ADVANCED EDA (6) AVERAGE MFCC PER CLASS")
print("=====")

# --- Computing and plotting average MFCC per class ---
for lab in sorted(df["label"].unique()):
    sub_lab = df[df["label"] == lab]
    if len(sub_lab) == 0:
        continue
    take_n = min(AVG_MFCC_PER_CLASS_N, len(sub_lab))
    sub_samp = sub_lab.sample(take_n, random_state=0)

    mfcc_list = []
    for p in sub_samp["file_path"]:
        y, sr = safe_load(p)
        if y is None:
            continue

```

```

y, _ = librosa.effects.trim(y, top_db=TOP_DB_TRIM)
mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=N_MFCC)
mfcc_list.append(mfcc)

if len(mfcc_list) == 0:
    continue

# Aligning MFCC time axes by truncating to minimum length
min_T = min(m.shape[1] for m in mfcc_list)
mfcc_stack = np.stack([m[:, :min_T] for m in mfcc_list], axis=0)
avg_mfcc = np.mean(mfcc_stack, axis=0)

plt.figure(figsize=(7, 4))
librosa.display.specshow(avg_mfcc, x_axis="time", sr=sr)
plt.colorbar()
plt.title(f"Average MFCC (n={N_MFCC}) - {lab} (n={len(mfcc_list)})")
plt.tight_layout()
savefig(f"avg_mfcc_{lab.replace(' ', '_').lower()}.png")
plt.show()

# =====
# SAVING OUTPUT TABLES
# =====

print("\n=====")
print("SAVING OUTPUT TABLES")
print("=====")

class_counts.to_csv(EDA_OUTPUT_DIR / "class_counts.csv", index=False)
df_stats.to_csv(EDA_OUTPUT_DIR / "audio_stats_with_spectral_sample.csv", index=False)

if "dataset_source" in df.columns:
    pd.crosstab(df["dataset_source"], df["label"]).to_csv(EDA_OUTPUT_DIR / "source_vs_label.csv")
if "device_type" in df.columns:
    pd.crosstab(df["device_type"], df["label"]).to_csv(EDA_OUTPUT_DIR / "device_vs_label.csv")

print("Saved key tables:")
print("-", EDA_OUTPUT_DIR / "class_counts.csv")
print("-", EDA_OUTPUT_DIR / "audio_stats_with_spectral_sample.csv")
if (EDA_OUTPUT_DIR / "weird_samples.csv").exists():
    print("-", EDA_OUTPUT_DIR / "weird_samples.csv")
if (EDA_OUTPUT_DIR / "pairwise_ttests.csv").exists():
    print("-", EDA_OUTPUT_DIR / "pairwise_ttests.csv")
if (EDA_OUTPUT_DIR / "rf_feature_importance.csv").exists():
    print("-", EDA_OUTPUT_DIR / "rf_feature_importance.csv")
if (EDA_OUTPUT_DIR / "source_vs_label.csv").exists():
    print("-", EDA_OUTPUT_DIR / "source_vs_label.csv")
if (EDA_OUTPUT_DIR / "device_vs_label.csv").exists():
    print("-", EDA_OUTPUT_DIR / "device_vs_label.csv")

print("\nFull EDA complete. All plots and CSVs saved in:", EDA_OUTPUT_DIR)

```

```

Base directory : C:\Users\chila\Downloads\EDA - AISD Project Work
Metadata       : C:\Users\chila\Downloads\EDA - AISD Project Work\metadata_handover.csv
Audio directory: C:\Users\chila\Downloads\EDA - AISD Project Work\audio_files
Outputs        : C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs
Audio files found: 1211
Example files: ['C:\\Users\\chila\\Downloads\\EDA - AISD Project Work\\audio_files\\asthma_v2_asthma_P10AsthmaIE_49.wav', 'C:\\Users\\chila\\Downloads\\EDA - AISD Project Work\\audio_files\\asthma_v2_asthma_P10AsthmaIU_46.wav', 'C:\\Users\\chila\\Downloads\\EDA - AISD Project Work\\audio_files\\asthma_v2_asthma_P10AsthmaIU_50.wav']

```

```

Loaded metadata: (1211, 5)
Columns: ['file_id', 'label', 'source', 'age', 'gender']
          file_id      label \
0  asthma_v2_Bronchial_P21BronchialTc_101.wav  Bronchitis
1  asthma_v2_Bronchial_P17BronchialTc_81.wav   Bronchitis
2  asthma_v2_Bronchial_P21BronchialSc_104.wav  Bronchitis

```

```

          source  age  gender
0  Asthma_V2_Smartphone  NaN    NaN
1  Asthma_V2_Smartphone  NaN    NaN
2  Asthma_V2_Smartphone  NaN    NaN
Created 'file_path' from 'file_id'.

```

```

Missing audio files: 0 / 1211
After removing missing files: (1211, 7)

```

```

=====
BASIC EDA (1) DATASET STATISTICS
=====
Total samples: 1211
Classes: ['Bronchitis', 'asthma', 'copd', 'healthy', 'pneumonia']

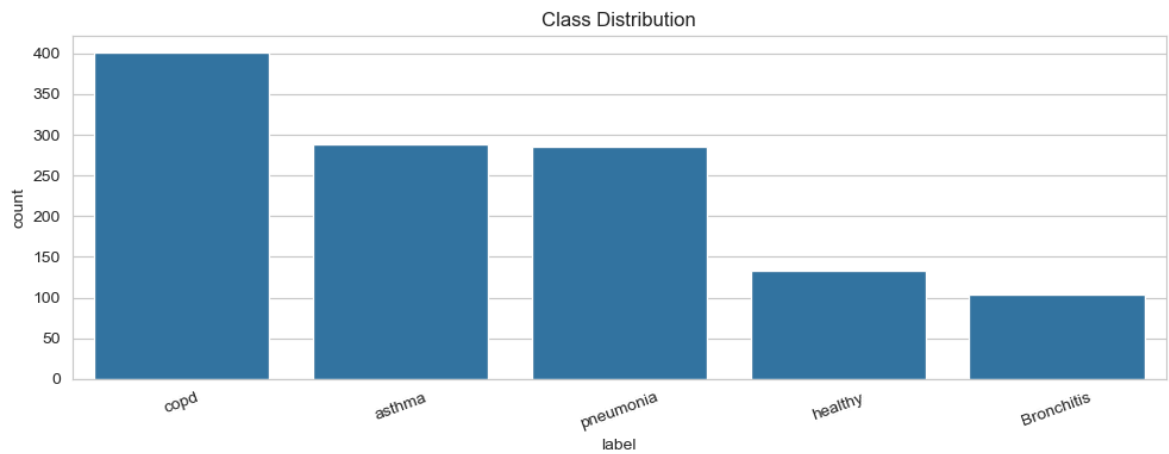
```

	label	count
0	copd	401
1	asthma	288
2	pneumonia	285
3	healthy	133
4	Bronchitis	104

```

Imbalance ratio (max/min): 3.86
Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\class_distribution.png

```



=====

BASIC EDA (2) AUDIO-LEVEL STATS

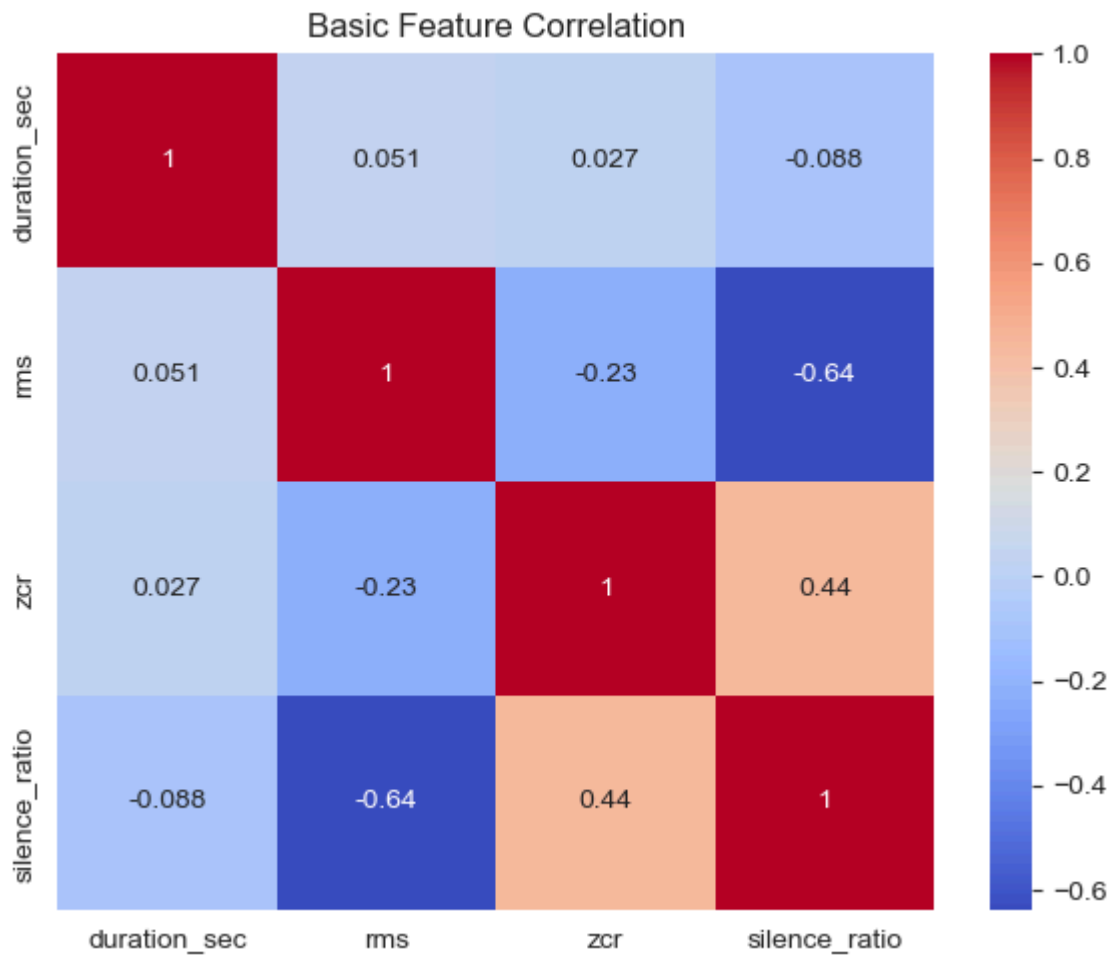
=====

Computing basic audio stats: 0% | 0/1211 [00:00<?, ?it/s]

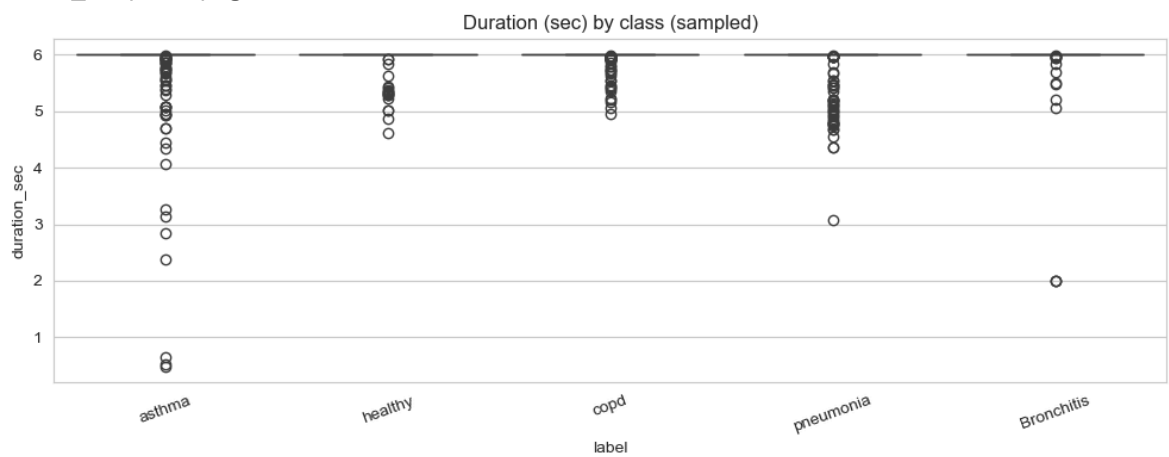
Bad/failed loads: 0/1211

	duration_sec	rms	zcr	silence_ratio
count	1211.000000	1211.000000	1211.000000	1211.000000
mean	5.888652	0.182868	0.005212	0.109366
std	0.455199	0.128094	0.005168	0.120540
min	0.480000	0.009244	0.000830	0.011062
25%	6.000000	0.079247	0.002512	0.036297
50%	6.000000	0.152658	0.003480	0.061689
75%	6.000000	0.260564	0.006211	0.134391
max	6.000000	0.709087	0.057355	0.783635

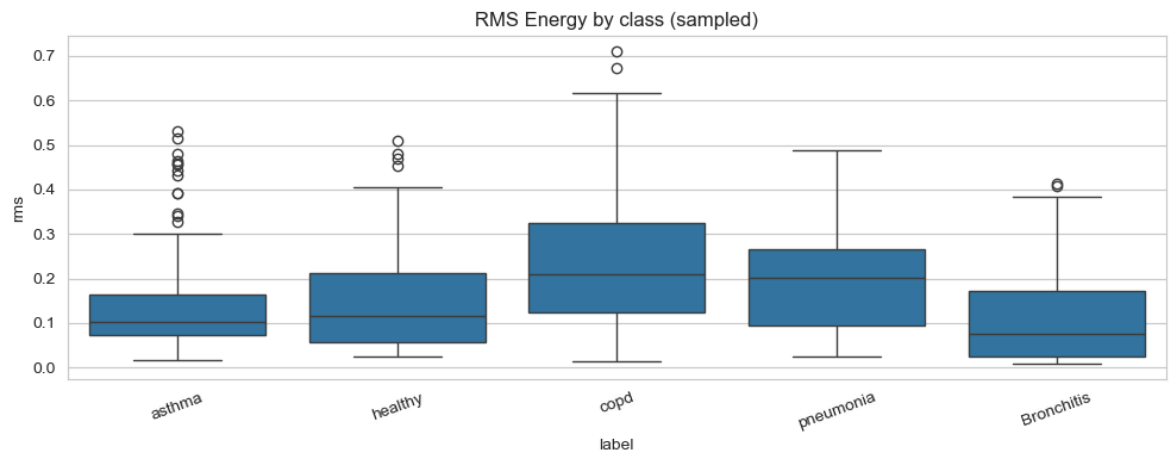
Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\basic_feature_correlation.png



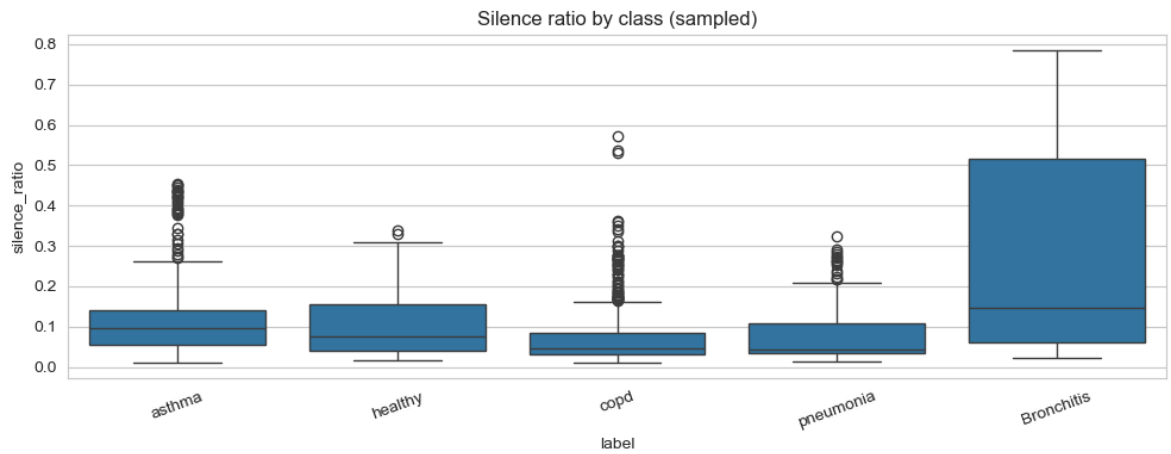
Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\duration_by_class_boxplot.png



Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\rms_by_class_boxplot.png



Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\silence_ratio_by_class_boxplot.png



Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\duration_vs_rms_scatter.png

	file_path	label	rms	duration_sec	silence_ratio	reason
0	C:\Users\chila\Downloads\EDA - AISD Project Wo...	asthma	0.125150	0.480000	0.050781	Shortes duration
1	C:\Users\chila\Downloads\EDA - AISD Project Wo...	asthma	0.073922	0.528000	0.136127	Shortes duration
2	C:\Users\chila\Downloads\EDA - AISD Project Wo...	asthma	0.099677	0.656000	0.075743	Shortes duration
3	C:\Users\chila\Downloads\EDA - AISD Project Wo...	Bronchitis	0.023842	2.000062	0.325427	Shortes duration
4	C:\Users\chila\Downloads\EDA - AISD Project Wo...	Bronchitis	0.020865	2.000062	0.563545	Shortes duration
5	C:\Users\chila\Downloads\EDA - AISD Project Wo...	asthma	0.081094	6.000000	0.116854	Longes duration
6	C:\Users\chila\Downloads\EDA - AISD Project Wo...	healthy	0.096230	6.000000	0.074438	Longes duration
7	C:\Users\chila\Downloads\EDA - AISD Project Wo...	copd	0.126038	6.000000	0.068812	Longes duration
8	C:\Users\chila\Downloads\EDA - AISD Project Wo...	pneumonia	0.254104	6.000000	0.030937	Longes duration
9	C:\Users\chila\Downloads\EDA - AISD Project Wo...	copd	0.308814	6.000000	0.036687	Longes duration
10	C:\Users\chila\Downloads\EDA - AISD Project Wo...	Bronchitis	0.009244	6.000000	0.783635	Quietes (lov RMS)
11	C:\Users\chila\Downloads\EDA - AISD Project Wo...	Bronchitis	0.009630	6.000000	0.754604	Quietes (lov RMS)
12	C:\Users\chila\Downloads\EDA - AISD Project Wo...	Bronchitis	0.010324	6.000000	0.738083	Quietes (lov RMS)
13	C:\Users\chila\Downloads\EDA - AISD Project Wo...	Bronchitis	0.013541	2.000062	0.646949	Quietes (lov RMS)
14	C:\Users\chila\Downloads\EDA - AISD Project Wo...	Bronchitis	0.013557	6.000000	0.652521	Quietes (lov RMS)
15	C:\Users\chila\Downloads\EDA - AISD Project Wo...	copd	0.709087	6.000000	0.012594	Loudes (high RMS)
16	C:\Users\chila\Downloads\EDA - AISD Project Wo...	copd	0.673840	6.000000	0.014010	Loudes (high RMS)
17	C:\Users\chila\Downloads\EDA - AISD Project Wo...	copd	0.616956	5.968000	0.022663	Loudes (high RMS)

	file_path	label	rms	duration_sec	silence_ratio	reason
18	C:\Users\chila\Downloads\EDA - AISD Project Wo...	copd	0.591173	6.000000	0.011281	Louder (high RMS)
19	C:\Users\chila\Downloads\EDA - AISD Project Wo...	copd	0.590939	6.000000	0.011062	Louder (high RMS)
20	C:\Users\chila\Downloads\EDA - AISD Project Wo...	Bronchitis	0.009244	6.000000	0.783635	Moderate silence
21	C:\Users\chila\Downloads\EDA - AISD Project Wo...	Bronchitis	0.009630	6.000000	0.754604	Moderate silence
22	C:\Users\chila\Downloads\EDA - AISD Project Wo...	Bronchitis	0.010324	6.000000	0.738083	Moderate silence
23	C:\Users\chila\Downloads\EDA - AISD Project Wo...	Bronchitis	0.018619	6.000000	0.654896	Moderate silence
24	C:\Users\chila\Downloads\EDA - AISD Project Wo...	Bronchitis	0.013557	6.000000	0.652521	Moderate silence

Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\weird_samples.csv

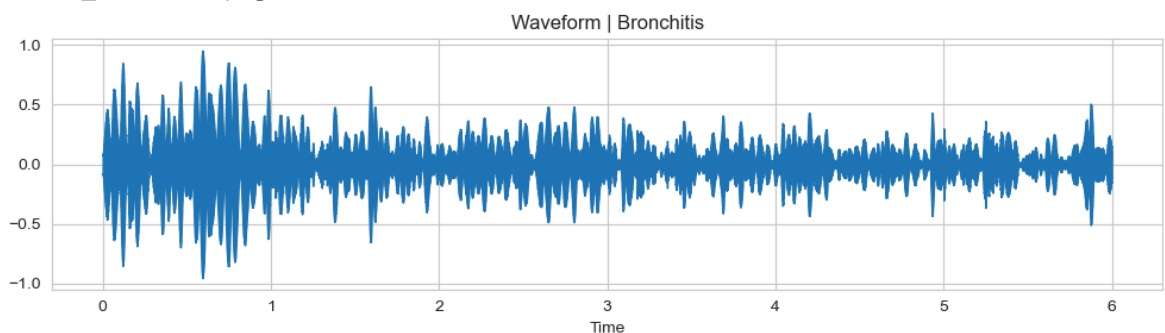
=====

BASIC EDA (3) EXAMPLE VISUALS PER CLASS

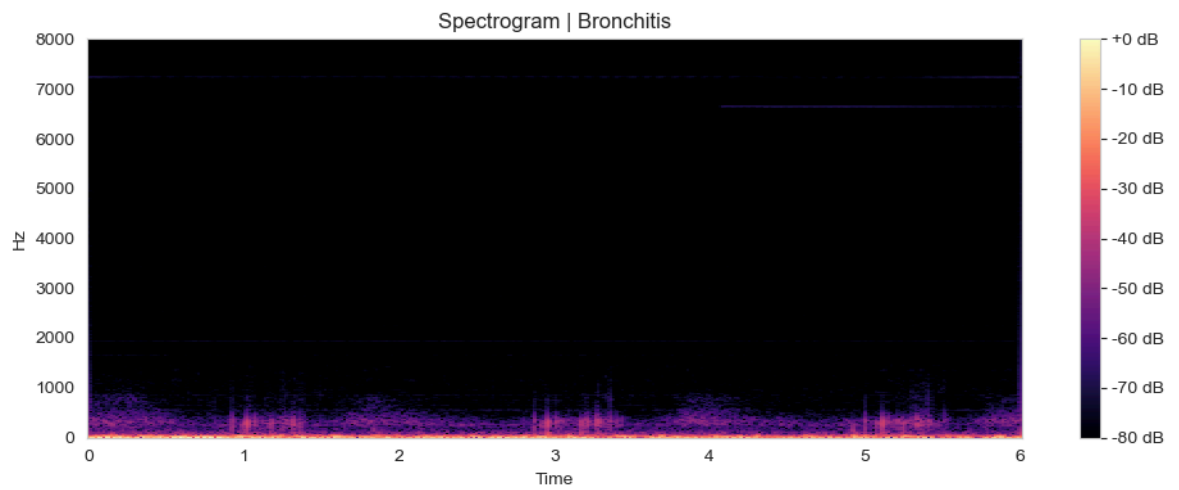
=====

=== Bronchitis === File: asthma_v2_Bronchial_P21BronchialSc_102.wav

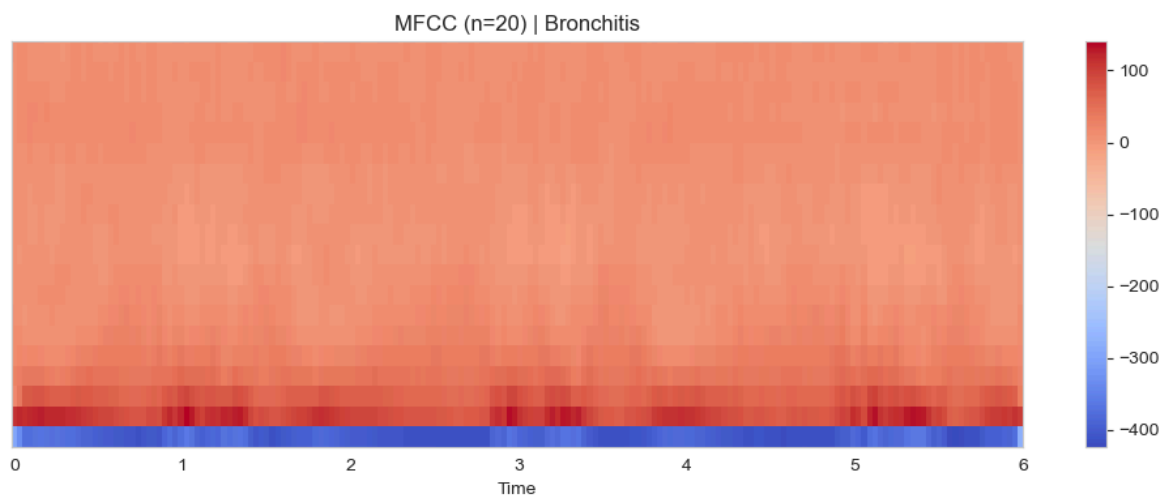
Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\example_bronchitis_waveform.png



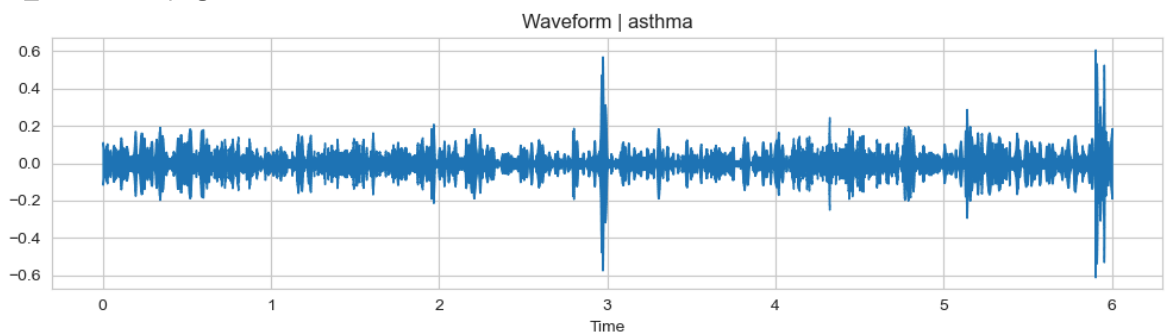
Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\example_bronchitis_spectrogram.png



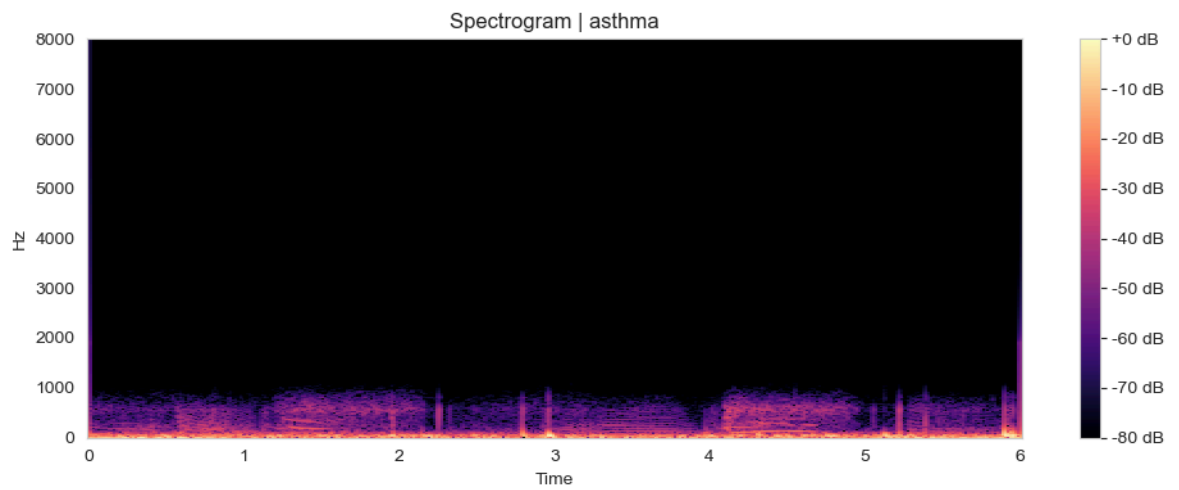
Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\example_bronchitis_mfcc.png



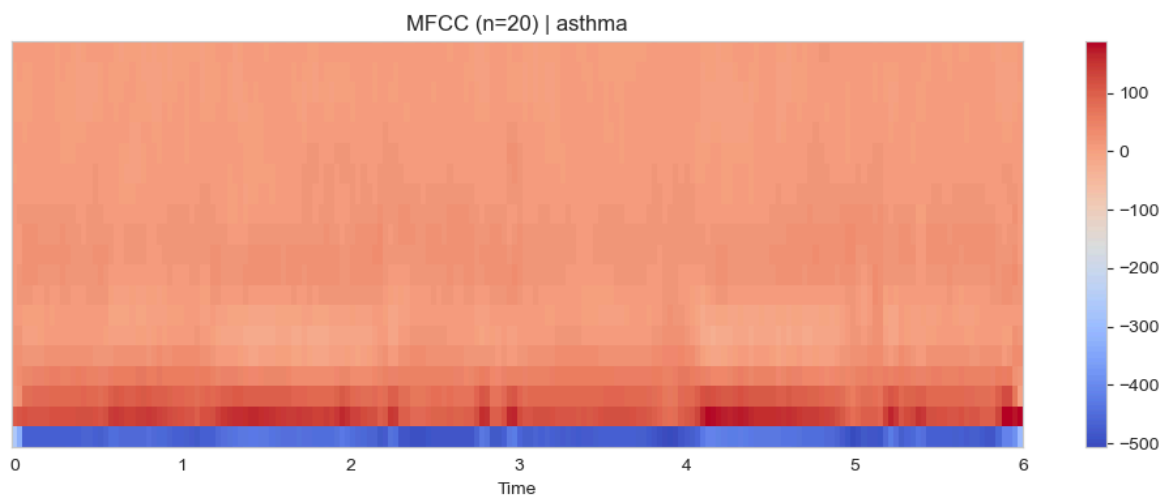
=== asthma === File: asthma_v2_asthma_P36WheezingIU_178.wav
 Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\example_asthma_waveform.png



Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\example_asthma_spectrogram.png

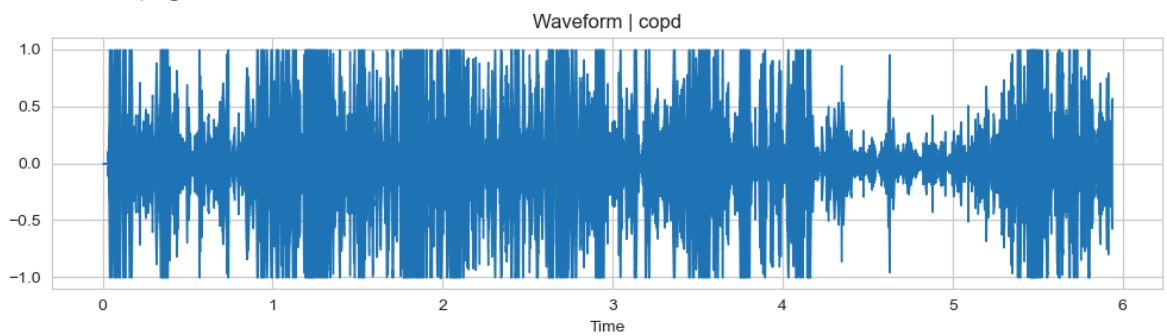


Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\example_asthma_mfcc.png

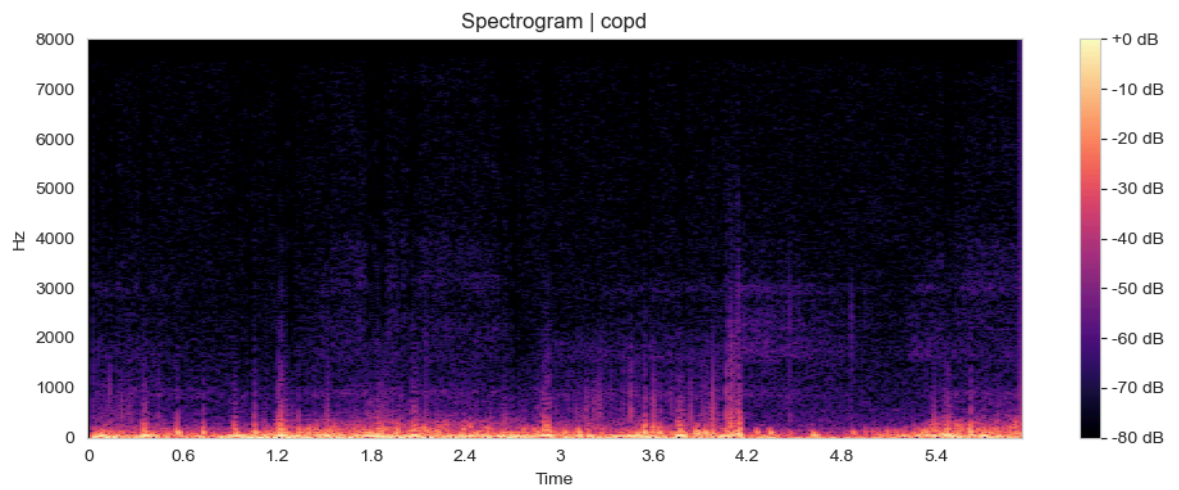


=== copd === File: asthma_v2_copd_P28COPDMc_224.wav

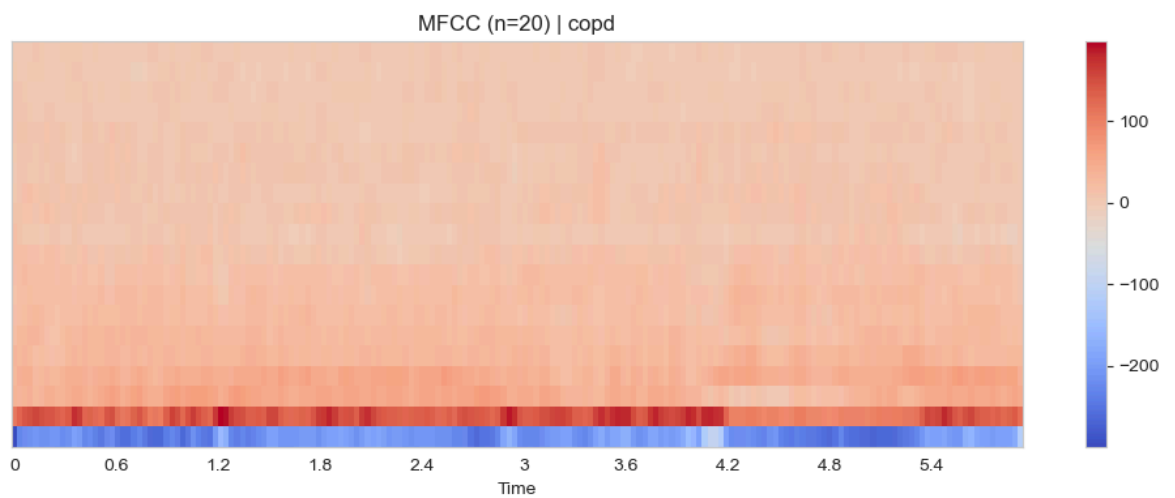
Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\example_copd_waveform.png



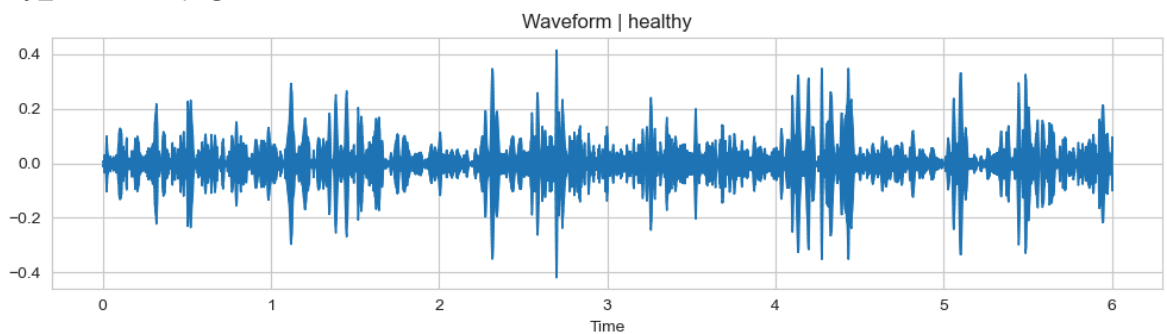
Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\example_copd_spectrogram.png



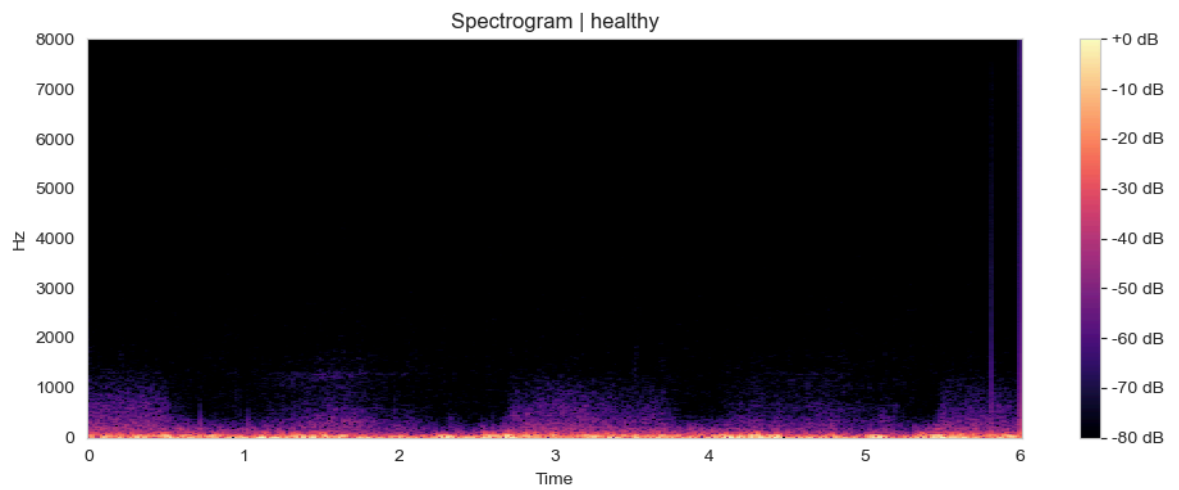
Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\example_copd_mfcc.png



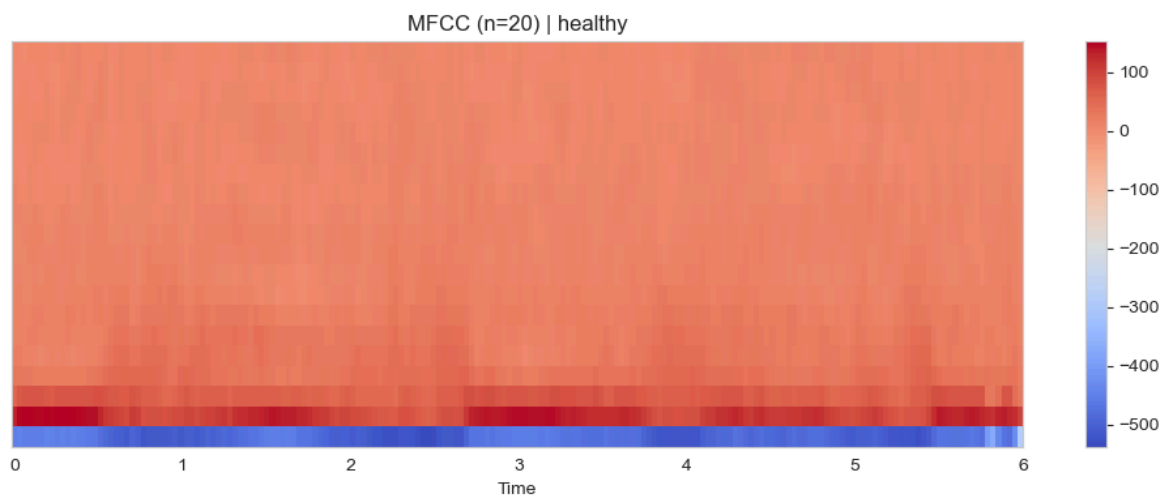
=== healthy === File: asthma_v2_healthy_P4Healthy22S.wav
 Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\example_healthy_waveform.png



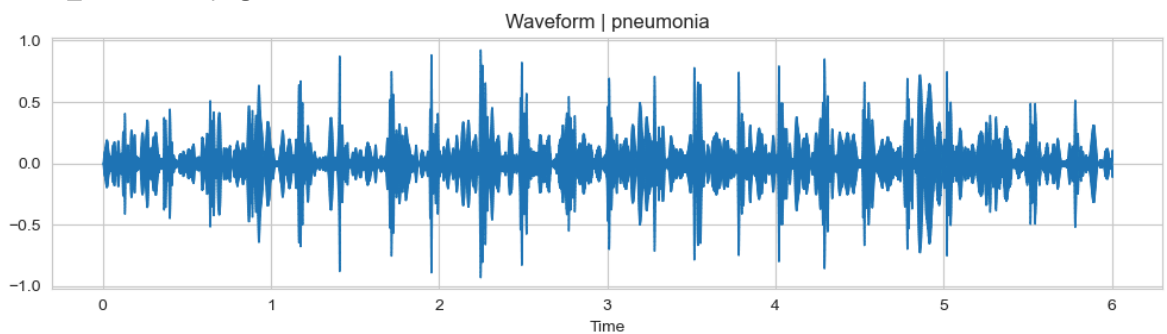
Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\example_healthy_spectrogram.png



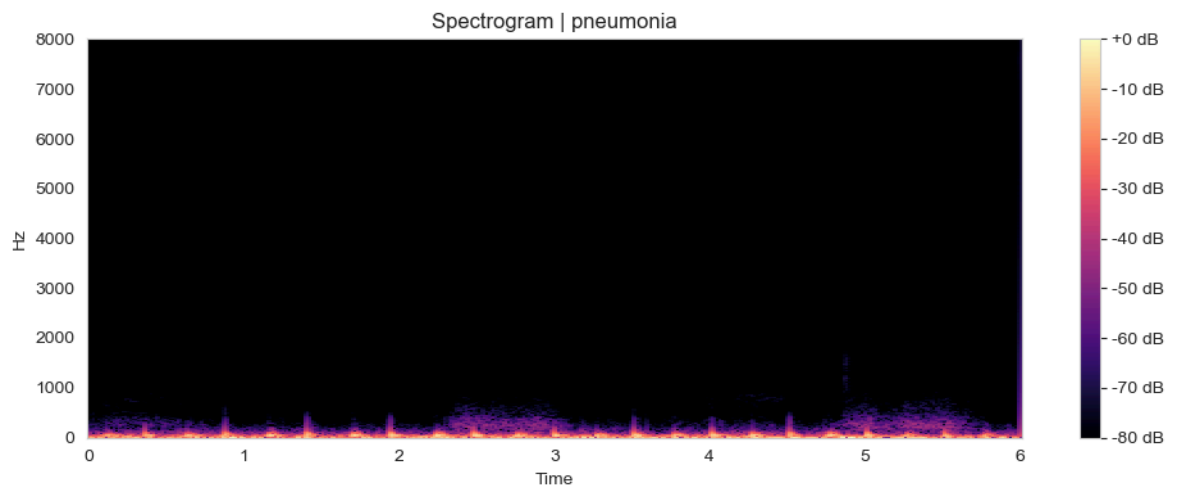
Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\example_healthy_mfcc.png



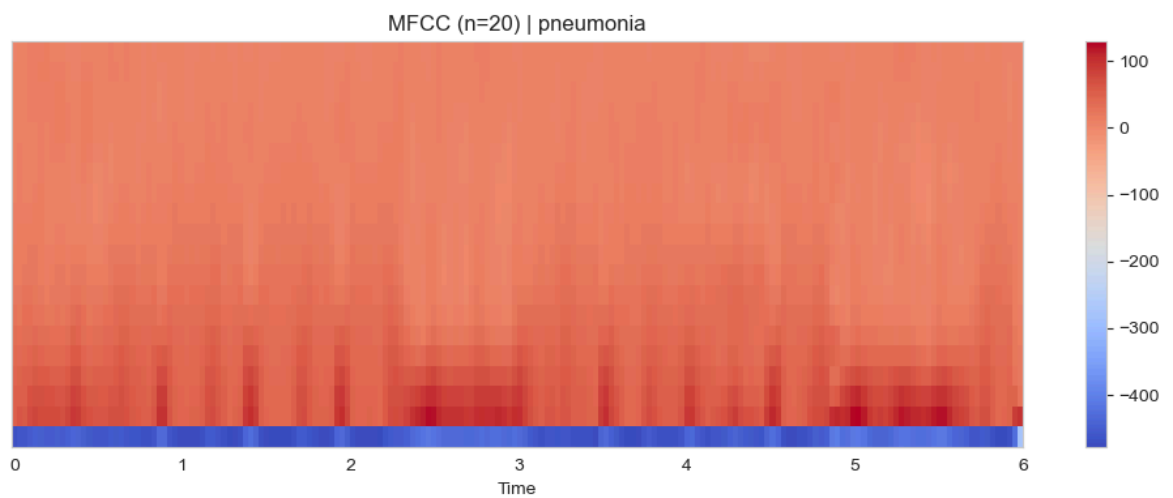
=== pneumonia === File: asthma_v2_pneumonia_P8Pneumonia23N.wav
 Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\example_pneumonia_waveform.png



Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\example_pneumonia_spectrogram.png



Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\example_pneumonia_mfcc.png



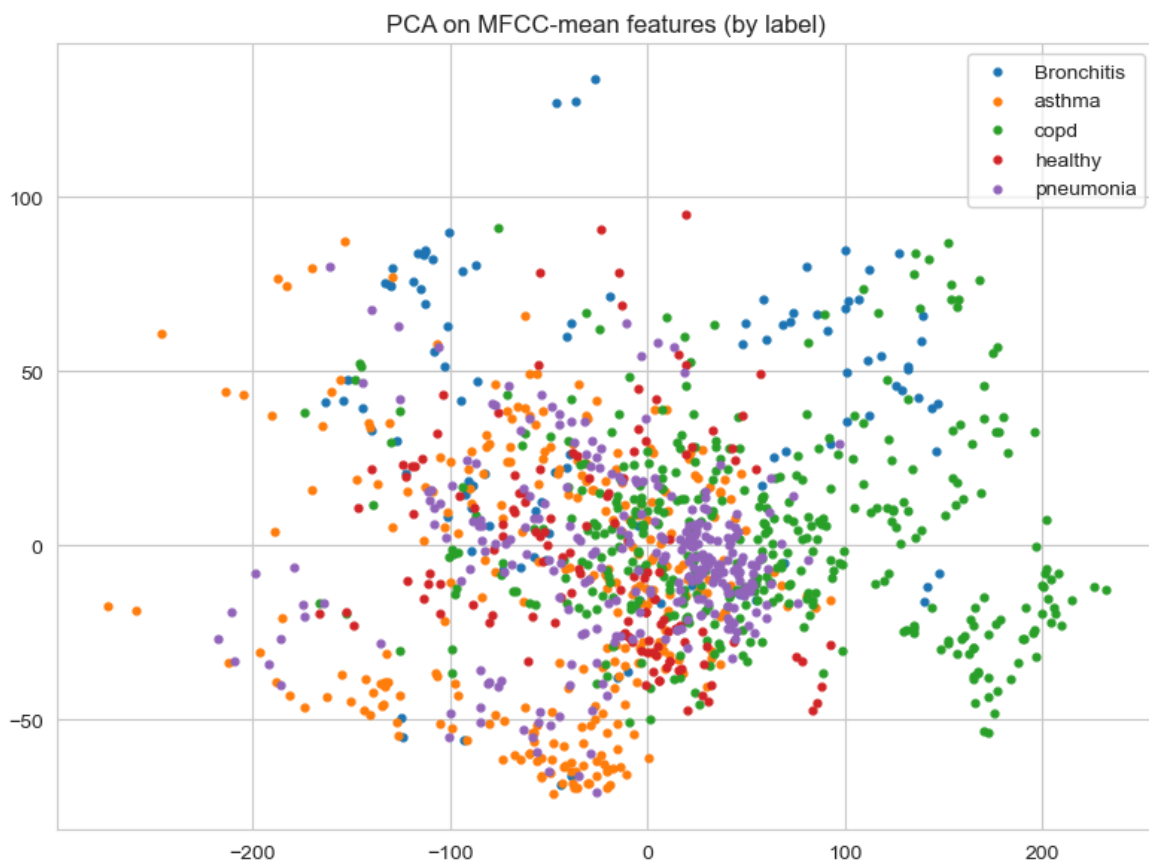
```
=====
BASIC EDA (4) DOMAIN SHIFT CHECKS
=====
```

```
=====
BASIC EDA (5) FEATURE EMBEDDINGS (MFCC-mean)
=====
```

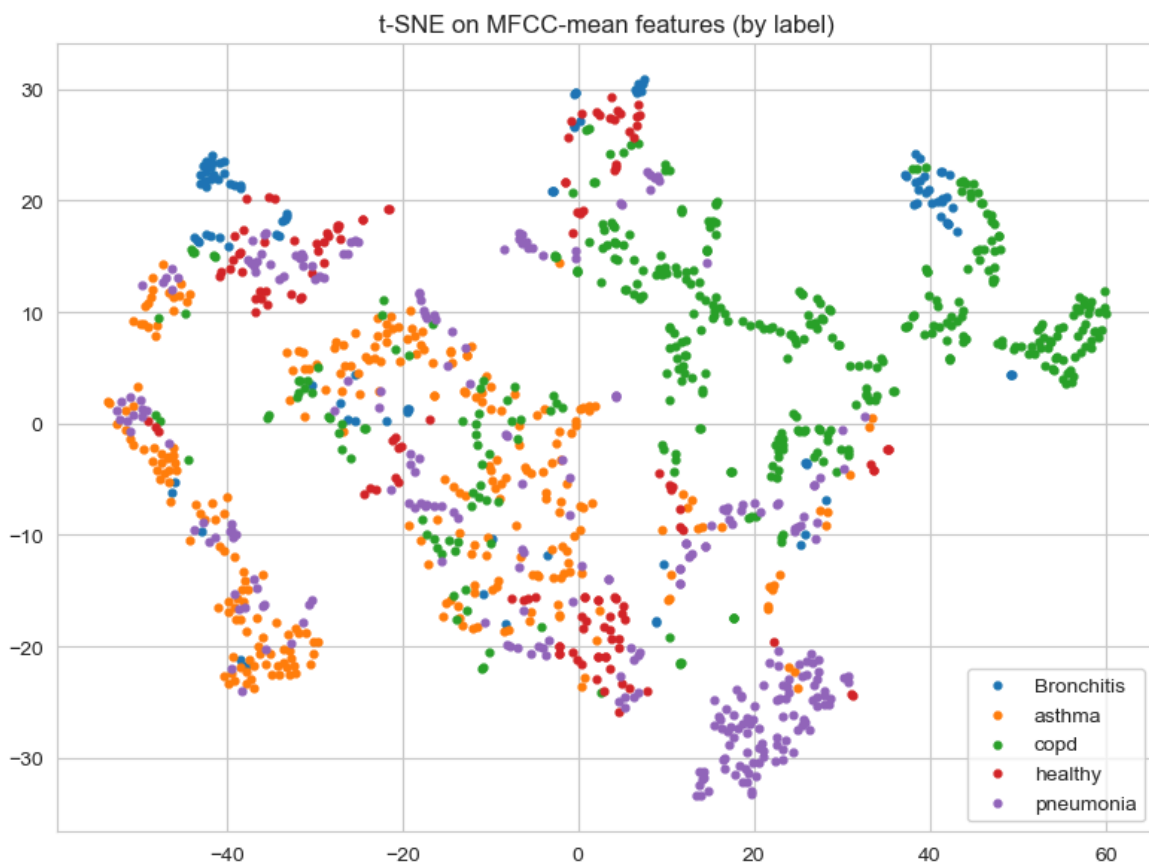
```
Extracting MFCC-mean features:  0%|          | 0/1200 [00:00<?, ?it/s]
```

```
Feature matrix: (1200, 20)
```

```
Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\pca_mfcc_mean_by_label.png
```



Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\tsne_mfcc_mean_by_label.png



=====

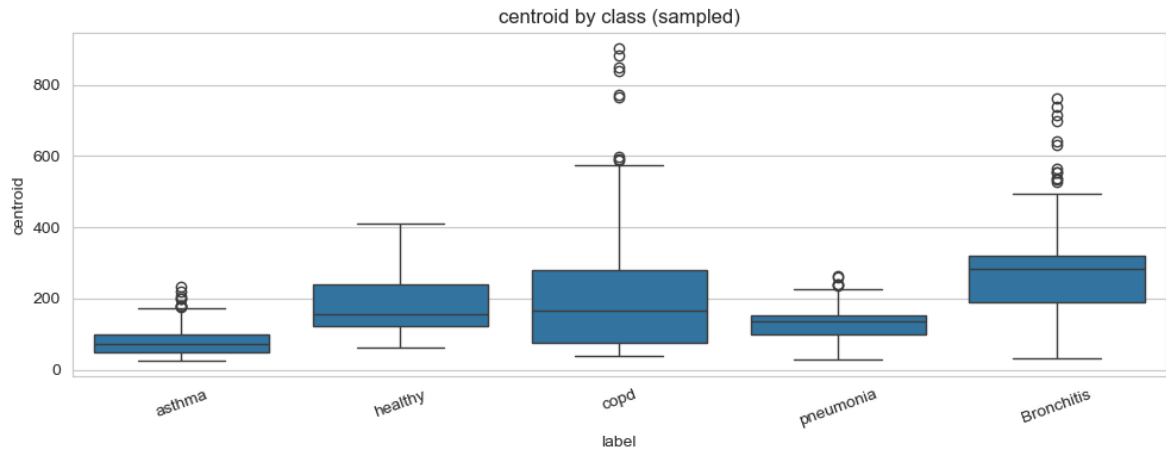
ADVANCED EDA (1) SPECTRAL FEATURES

=====

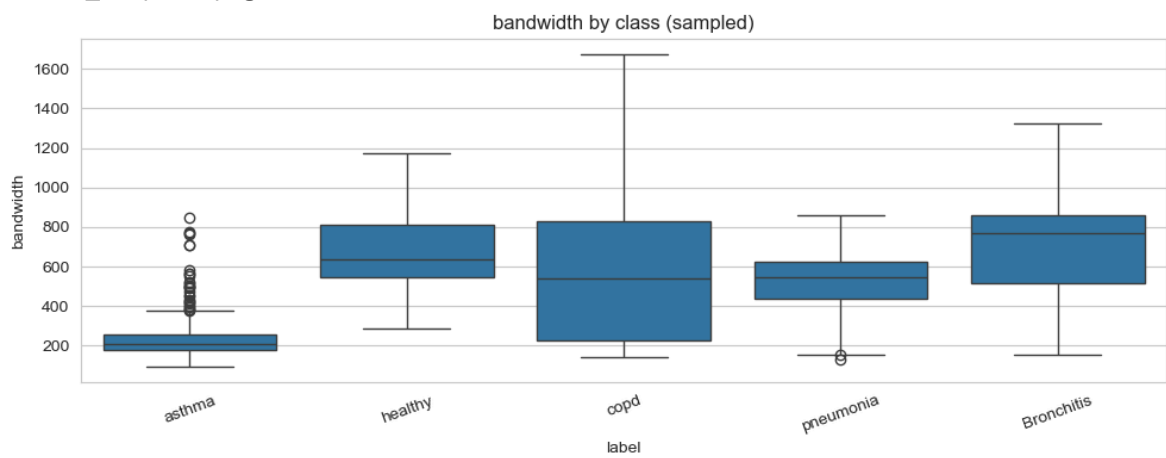
Computing spectral features: 0% | 0/1211 [00:00<?, ?it/s]

Added spectral features to df_stats.

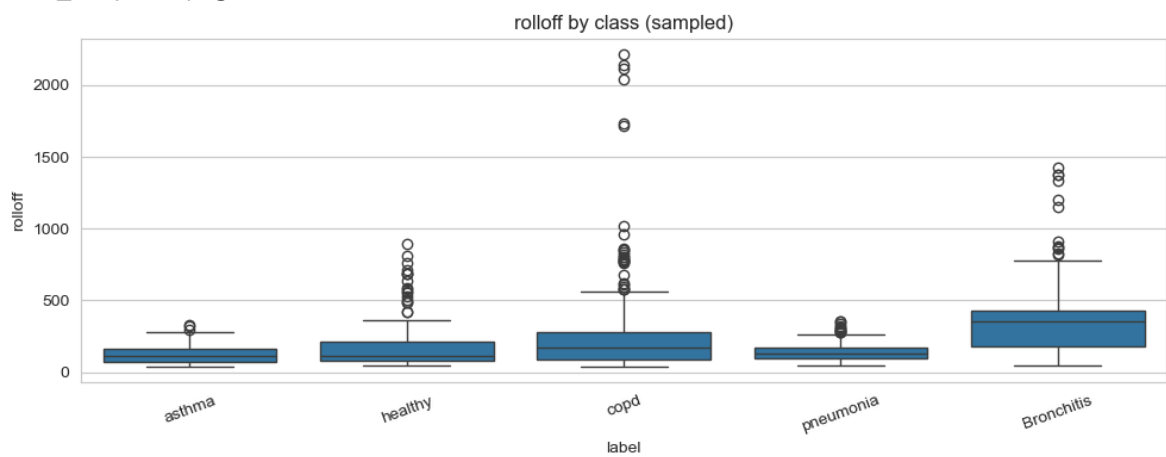
Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\centroid_by_class_boxplot.png



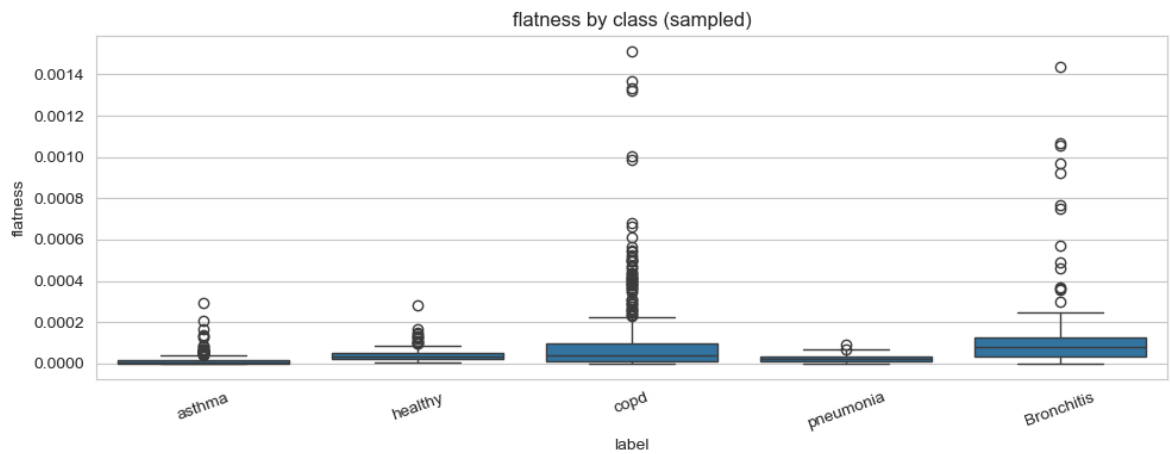
Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\bandwidth_by_class_boxplot.png



Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\rolloff_by_class_boxplot.png



Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\flatness_by_class_boxplot.png

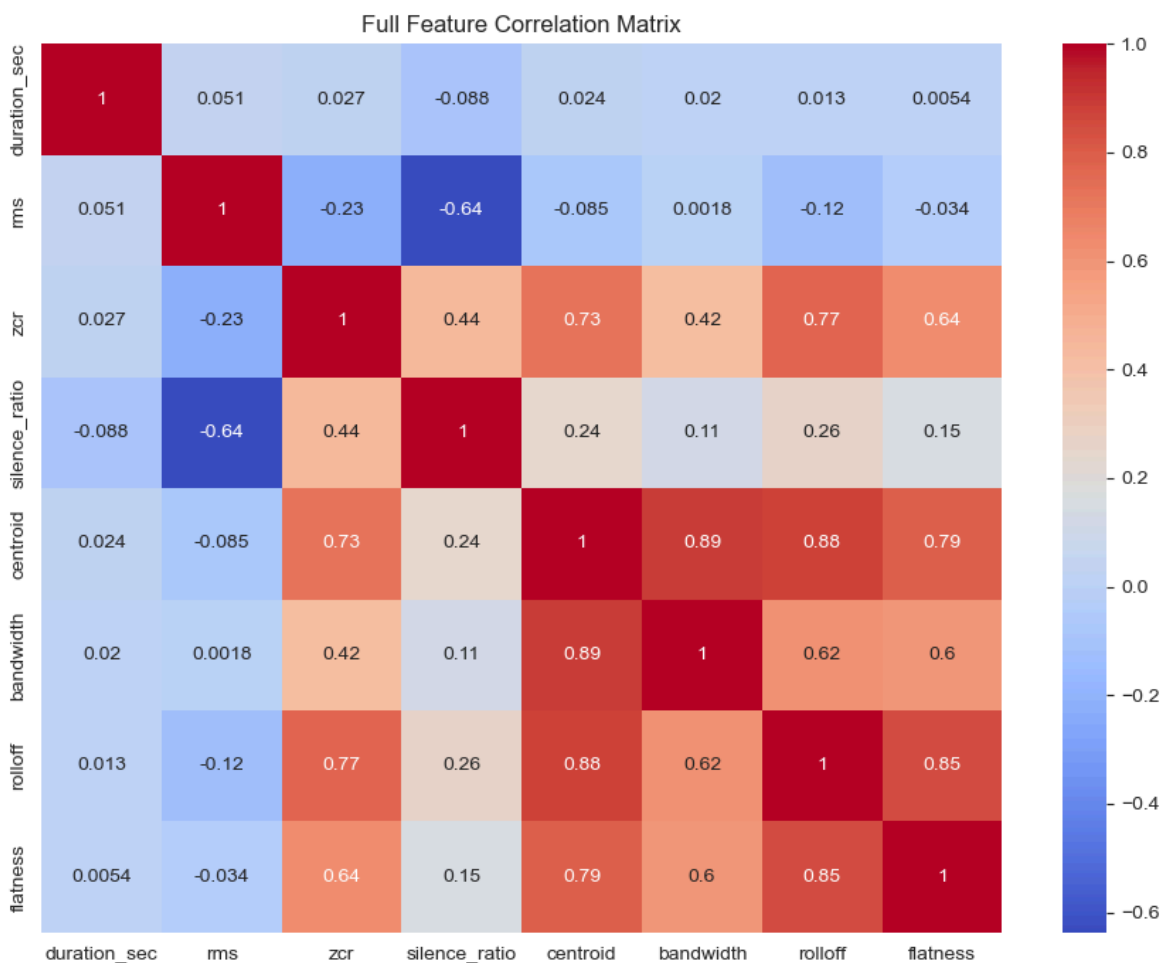


=====

ADVANCED EDA (2) FULL FEATURE CORRELATION

=====

Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\full_feature_correlation_matrix.png



=====

ADVANCED EDA (3) STATISTICAL TESTS (PAIRWISE)

=====

Testing labels : ['copd', 'asthma', 'pneumonia', 'healthy']

Testing features: ['rms', 'zcr', 'centroid', 'rolloff']

	class_A	class_B	feature	p_value
14	asthma	pneumonia	centroid	1.491510e-43
2	copd	asthma	centroid	8.609486e-43
5	copd	pneumonia	zcr	1.152591e-31
0	copd	asthma	rms	4.290010e-30
18	asthma	healthy	centroid	3.215895e-29
3	copd	asthma	rolloff	6.150421e-19
6	copd	pneumonia	centroid	4.936722e-17
1	copd	asthma	zcr	7.234311e-17
7	copd	pneumonia	rolloff	1.103407e-14
12	asthma	pneumonia	rms	3.479181e-13
22	pneumonia	healthy	centroid	2.288696e-11
8	copd	healthy	rms	7.829622e-11
9	copd	healthy	zcr	2.818181e-08
13	asthma	pneumonia	zcr	6.369830e-07
4	copd	pneumonia	rms	1.305543e-06
19	asthma	healthy	rolloff	1.893937e-05
15	asthma	pneumonia	rolloff	1.518927e-04
23	pneumonia	healthy	rolloff	1.447436e-03
20	pneumonia	healthy	rms	1.977840e-03
11	copd	healthy	rolloff	2.927981e-03
21	pneumonia	healthy	zcr	2.122602e-02
16	asthma	healthy	rms	3.109424e-02
10	copd	healthy	centroid	1.673574e-01
17	asthma	healthy	zcr	9.538754e-01

Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\pairwise_ttest.csv

=====

ADVANCED EDA (4) DOMAIN SHIFT FEATURE COMPARISON

=====

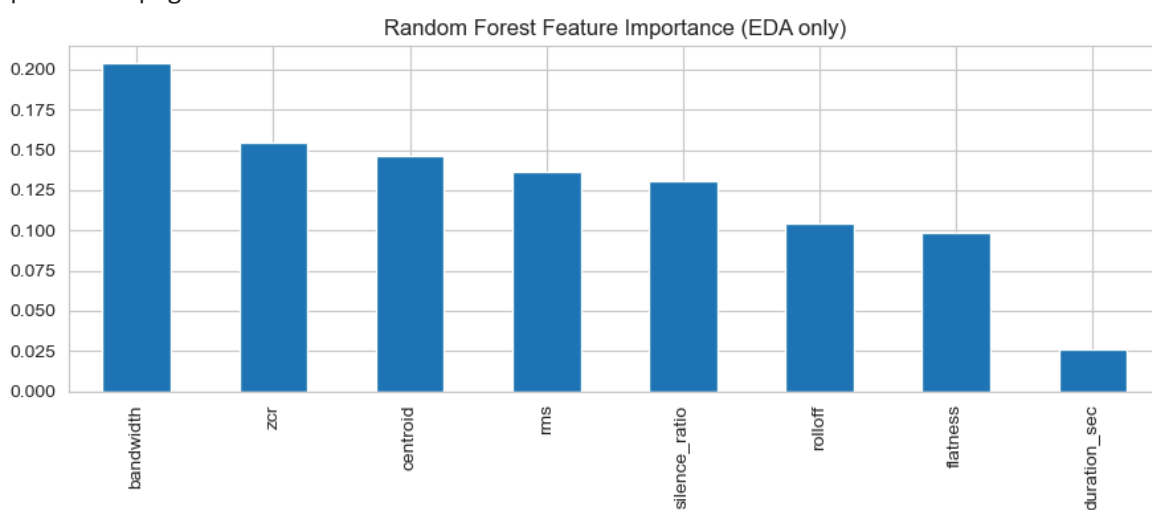
=====

ADVANCED EDA (5) QUICK FEATURE IMPORTANCE (RandomForest)

=====

	importance
bandwidth	0.204163
zcr	0.154207
centroid	0.145948
rms	0.136354
silence_ratio	0.130454
rolloff	0.104215
flatness	0.098293
duration_sec	0.026366

Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\rf_feature_importance.png



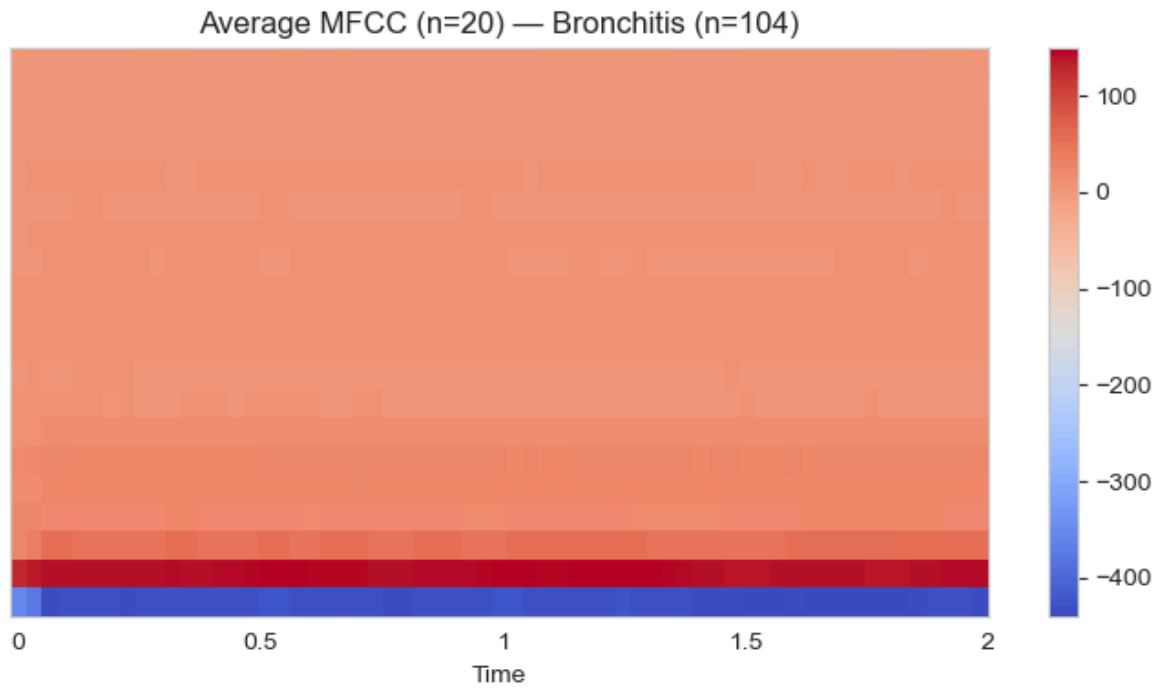
Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\rf_feature_importance.csv

=====

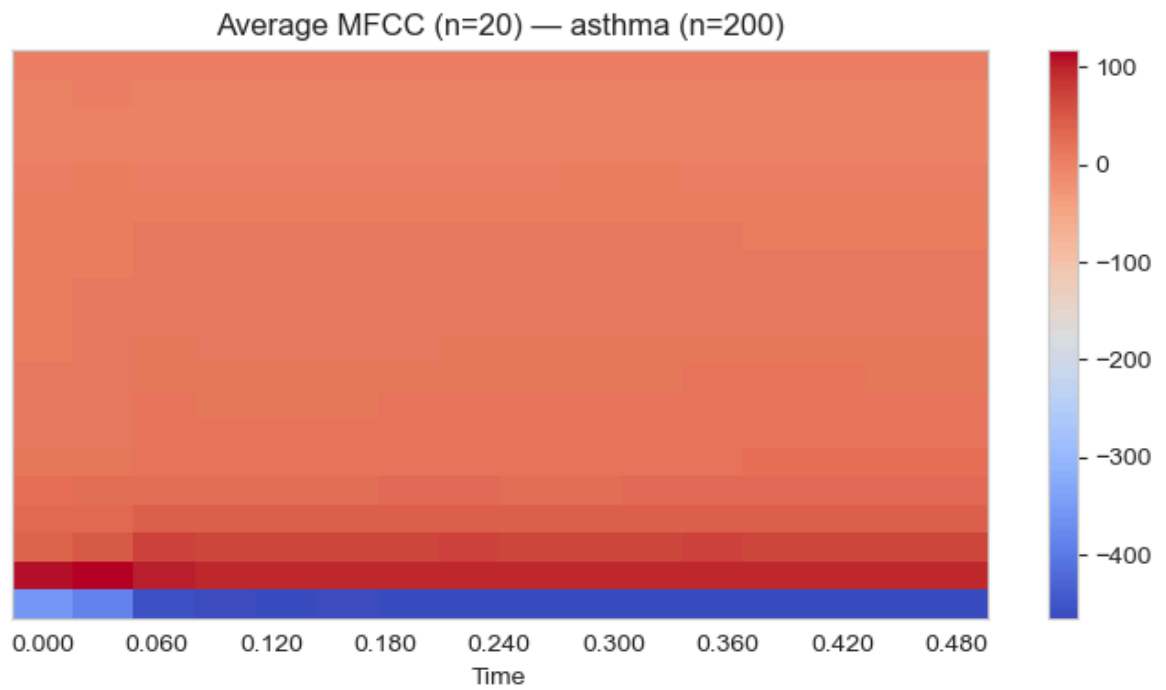
ADVANCED EDA (6) AVERAGE MFCC PER CLASS

=====

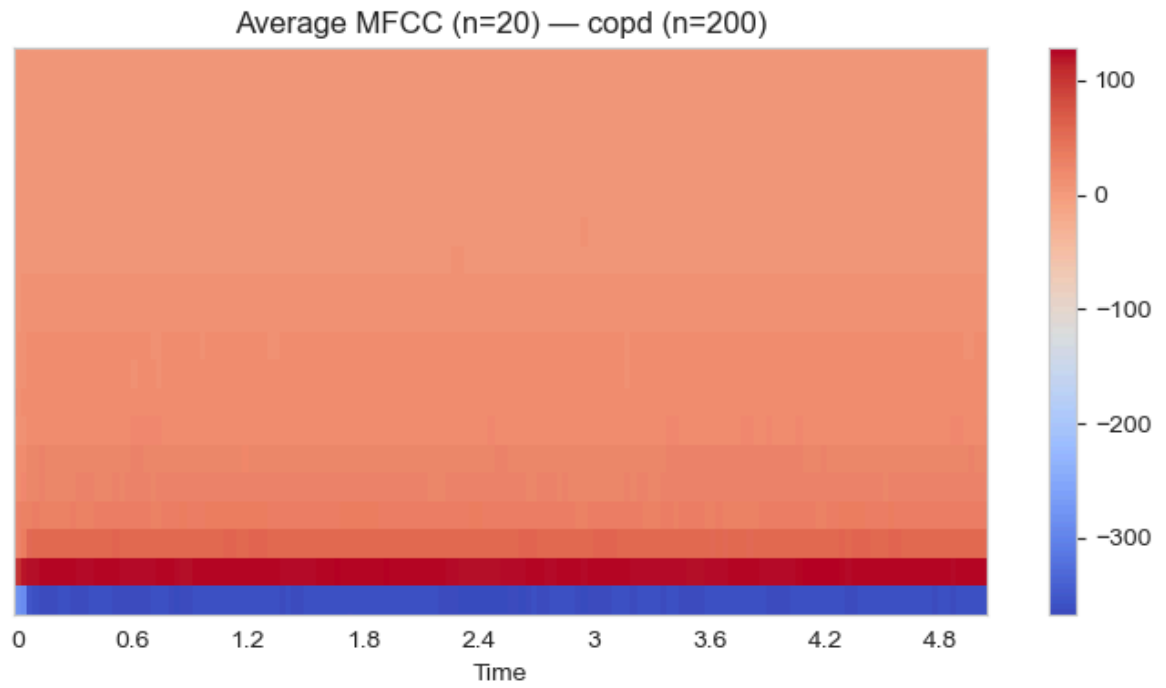
Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\avg_mfcc_bronchitis.png



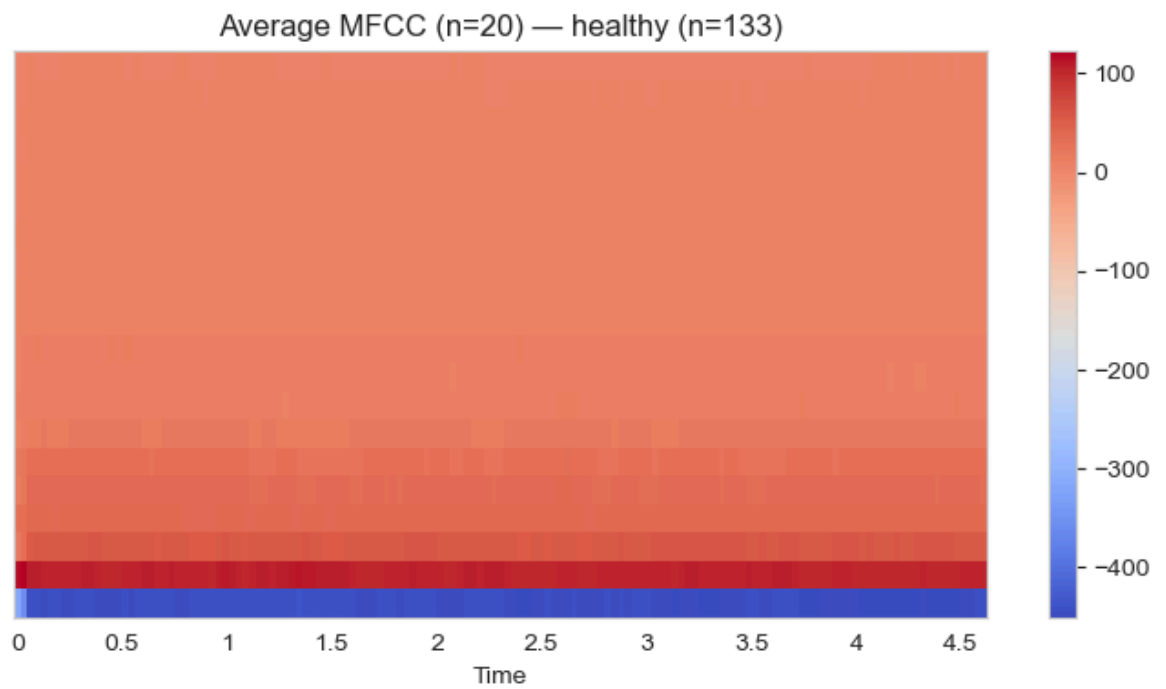
Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\avg_mfcc_asthma.png



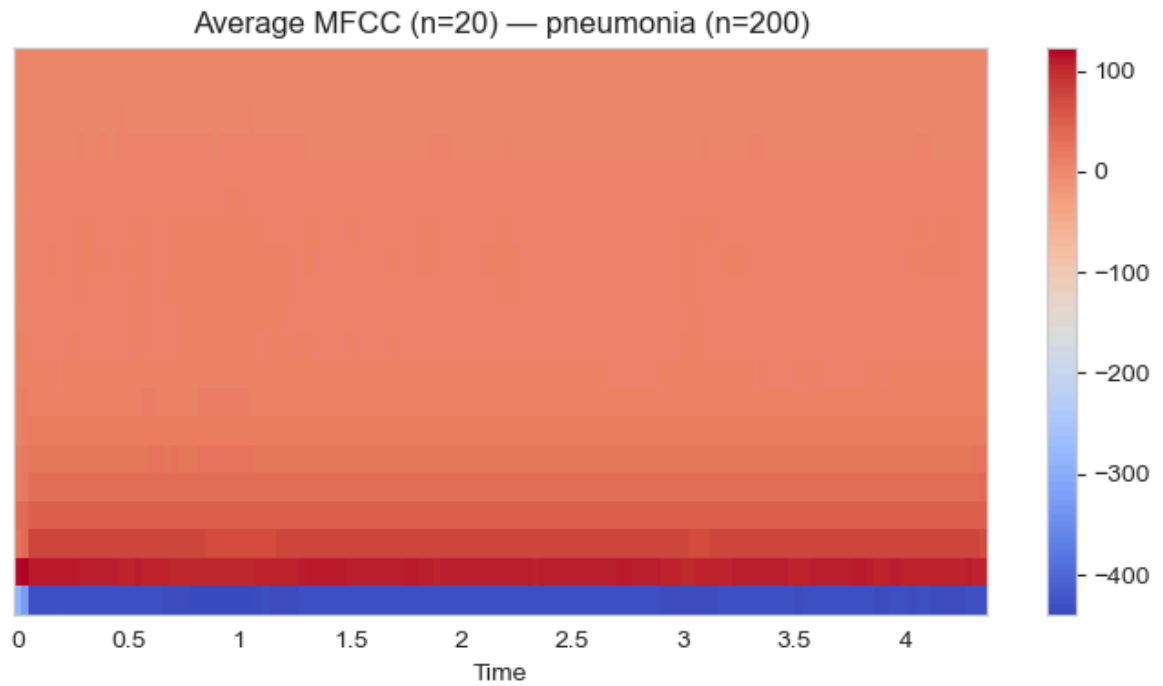
Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\avg_mfcc_copd.png



Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\avg_mfcc_healthy.png



Saved: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\avg_mfcc_pneumonia.png



=====

SAVING OUTPUT TABLES

=====

Saved key tables:

- C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\class_counts.csv
- C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\audio_stats_with_spectral_sample.csv
- C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\weird_samples.csv
- C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\pairwise_ttests.csv
- C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs\rf_feature_importance.csv

Full EDA complete. All plots and CSVs saved in: C:\Users\chila\Downloads\EDA - AISD Project Work\eda_outputs

In []: